

Informe De Diagramas de Componentes y Despliegue

Cipas

Juan Camilo Orjuela

Stid Vanegas

Luis David Rodríguez

Universidad del Tolima Sede Cat chaparral – Tolima

Programa ingeniería de sistemas IV

Ingeniería de software

Catedrático

Fabian Ávila orjuela

08/05/2026

En el ámbito de la Ingeniería de Software, cuando pasamos del diseño lógico a la ejecución técnica, nos encontramos con uno de los desafíos más importantes del ciclo de vida de desarrollo. Esta fase se conoce como implementación y no solo implica escribir código, sino también estructurar módulos funcionales y distribuirlos en infraestructuras tecnológicas complejas.

Para gestionar esta complejidad, el Lenguaje Unificado de Modelado, o UML, nos ofrece dos herramientas de visualización fundamentales: el Diagrama de Componentes y el Diagrama de Despliegue. Mientras que el primero se ocupa de la organización interna del software y sus dependencias, el segundo muestra cómo estos elementos interactúan con el hardware y los protocolos de red. A continuación, se detallarán ambos diagramas, analizando su simbología, aplicaciones prácticas y relevancia en el diseño de sistemas robustos y escalables.

2. Diagrama de Componentes: La Estructura de la Solución

El Diagrama de Componentes es una vista estática que describe la organización y las dependencias entre los componentes de software. Es esencial para implementar arquitecturas basadas en servicios o microservicios, ya que permite dividir un sistema grande en partes manejables.

Ejemplo de Estructura de Diagrama de Componentes

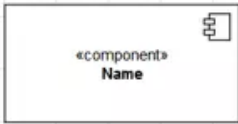
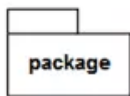
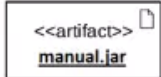
Elemento	Símbolo/notación	Explicación
Componente (<i>component</i>)		Símbolo para representar los módulos de un sistema (la interacción y la comunicación tienen lugar a través de interfaces).
Paquete		Un paquete combina varios elementos del sistema (por ejemplo, clases, componentes o interfaces) en un grupo.
Artefacto		Los artefactos son unidades físicas de información (por ejemplo, código fuente, archivos .exe, scripts o documentos) que se generan en el proceso de desarrollo o el tiempo de ejecución de un sistema o son necesarios para estos.
Interfaz ofrecida		Símbolo para una o más interfaces claramente definidas que proporcionan funciones, servicios o datos al mundo exterior (el semicírculo abierto también se denomina enchufe o <i>socket</i>).
Interfaz requerida		Símbolo de una interfaz necesaria para recibir funciones, servicios o datos del exterior (la notación del círculo con palo también se denomina <i>lollipop</i> o <i>piruleta</i>).
Puerto		Este símbolo indica un punto de interacción independiente entre un componente y su entorno.
Relación		Las líneas actúan como conectores e indican las relaciones entre los componentes.
Relación de dependencia		Conector especial para expresar una relación de dependencia entre los componentes del sistema (no siempre se indica).

Diagrama de componentes de una tienda online

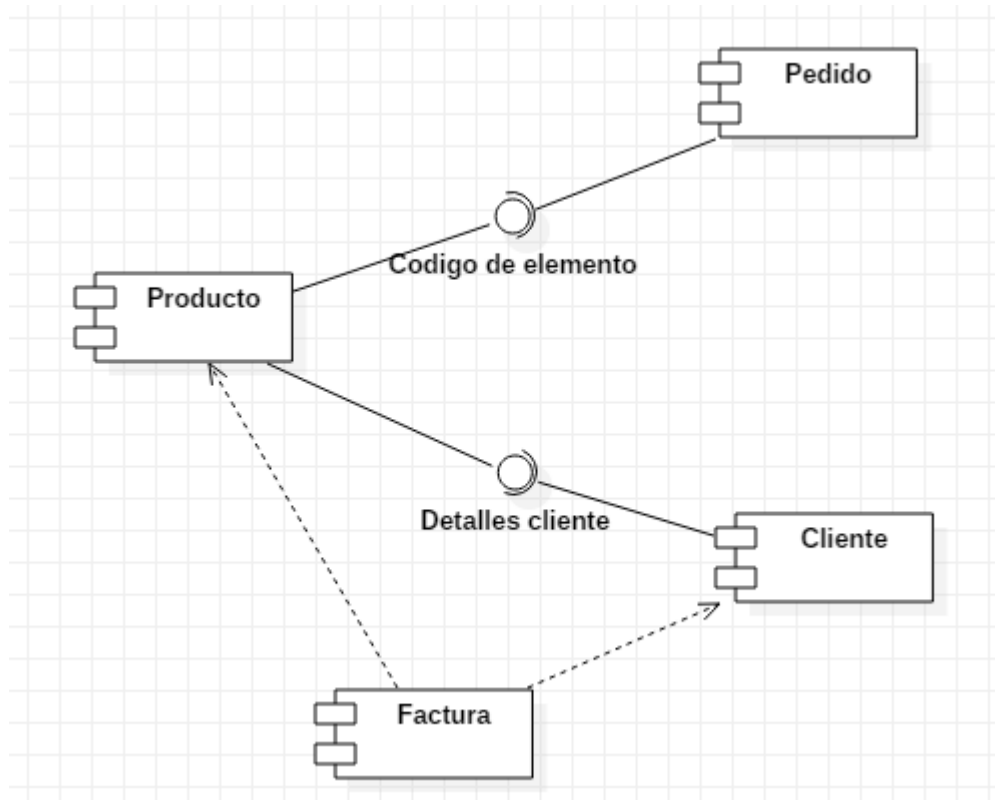
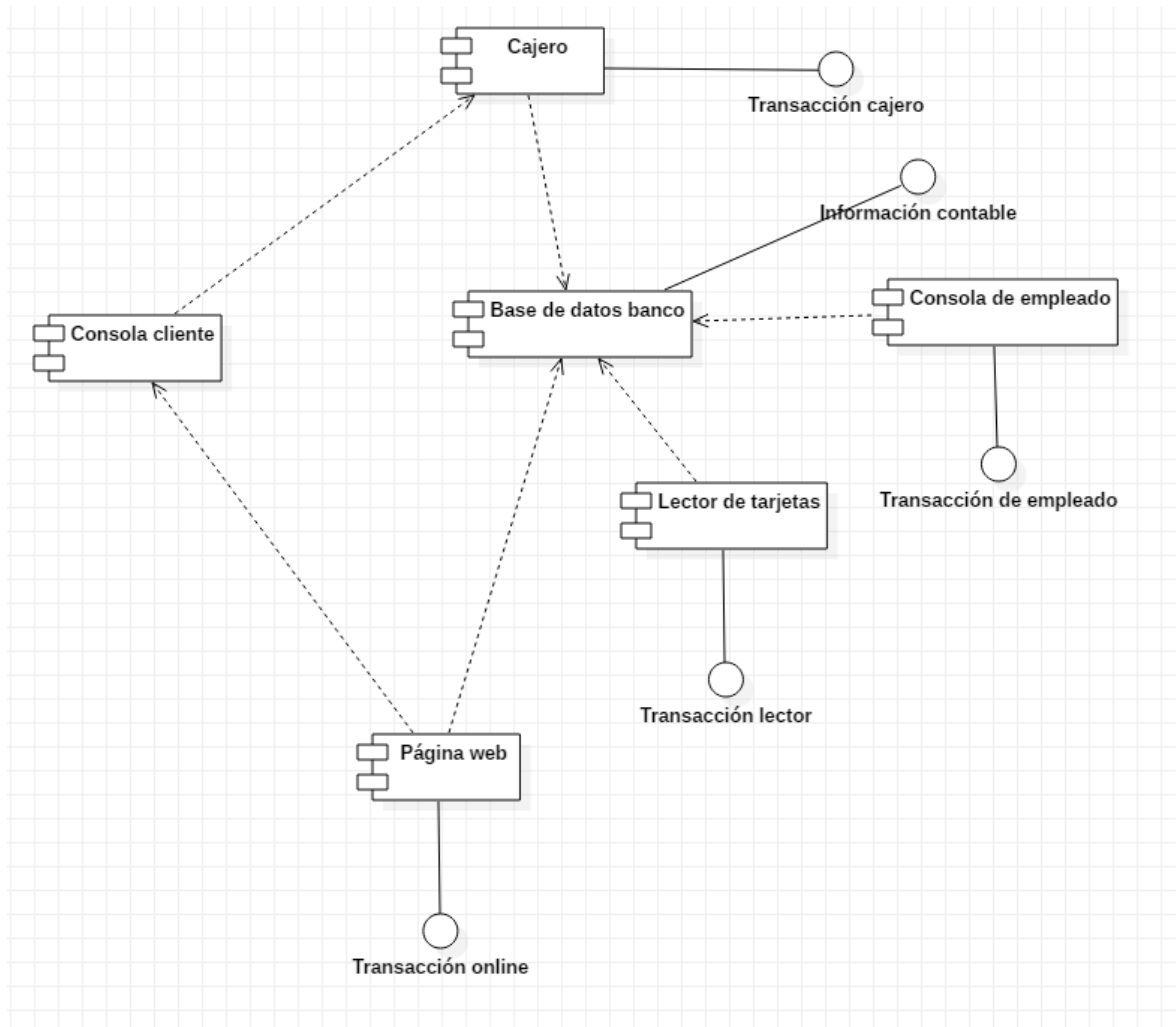


Diagrama de Componentes de un Banco



de Funcionalidad

Un componente es algo muy concreto. Puede ser un archivo de código, un programa que se puede ejecutar, una base de datos o un servicio en la web. Esto es diferente a las clases, que son más abstractas.

Interfaces como Contratos

Una interfaz es como un acuerdo entre diferentes partes. Hay dos tipos de interfaces importantes:

Interfaz Proporcionada: Esta es como la fachada de un componente. Define lo que el componente puede hacer por otros.

Interfaz Requerida: Esta especifica lo que un componente necesita de su entorno para funcionar correctamente. Esto permite que los componentes estén más separados y sea más fácil diseñar sistemas complejos.

✓ Estereotipos UML

Los estereotipos UML son una forma de clasificar los componentes. Algunos ejemplos incluyen:

-  <<subsystem>>: Un grupo de componentes que trabajan juntos.
-  <<module>>: Una unidad de software que realiza una tarea específica.
-  <<executable>>: Un programa que se puede ejecutar directamente.

2.1. Definición de Componente

Un componente es una unidad modular de un sistema que oculta su implementación detrás de un conjunto de interfaces externas. Un componente puede ser una base de datos, una biblioteca de vínculos dinámicos, un archivo JAR de Java, o un módulo de scripts en un servidor.

2.2. Elementos del Diagrama

- Interfaces Proporcionadas: Representadas por el símbolo de “piruleta”, indican que el componente ofrece una funcionalidad específica que otros pueden utilizar.
- Interfaces Requeridas: Representadas por un semicírculo, indican que el componente necesita un servicio externo para completar su lógica de negocio.
- Puertos: Son puntos de interacción entre el componente y su entorno, que agrupan interfaces relacionadas.
- Relaciones de Dependencia: Flechas discontinuas que conectan componentes, señalando que un cambio en un componente puede afectar a otro.

2.3. Aplicación en la Implementación

Este diagrama es vital para:

- Fomentar la reutilización de componentes en diferentes proyectos.
- Facilitar el mantenimiento, permitiendo sustituir un componente por otro sin afectar al resto del sistema.
- Gestionar la configuración, ayudando a los equipos de desarrollo a entender qué archivos y librerías deben estar presentes para que el sistema compile y enlace correctamente.

3. Diagrama de Despliegue: El Mapa de la Infraestructura

Si el diagrama de componentes nos dice qué construye el software, el Diagrama de Despliegue nos indica dónde reside y cómo se comunica. Este diagrama modela el hardware físico y el entorno de ejecución.

Ejemplos de diagramas de Despliegue

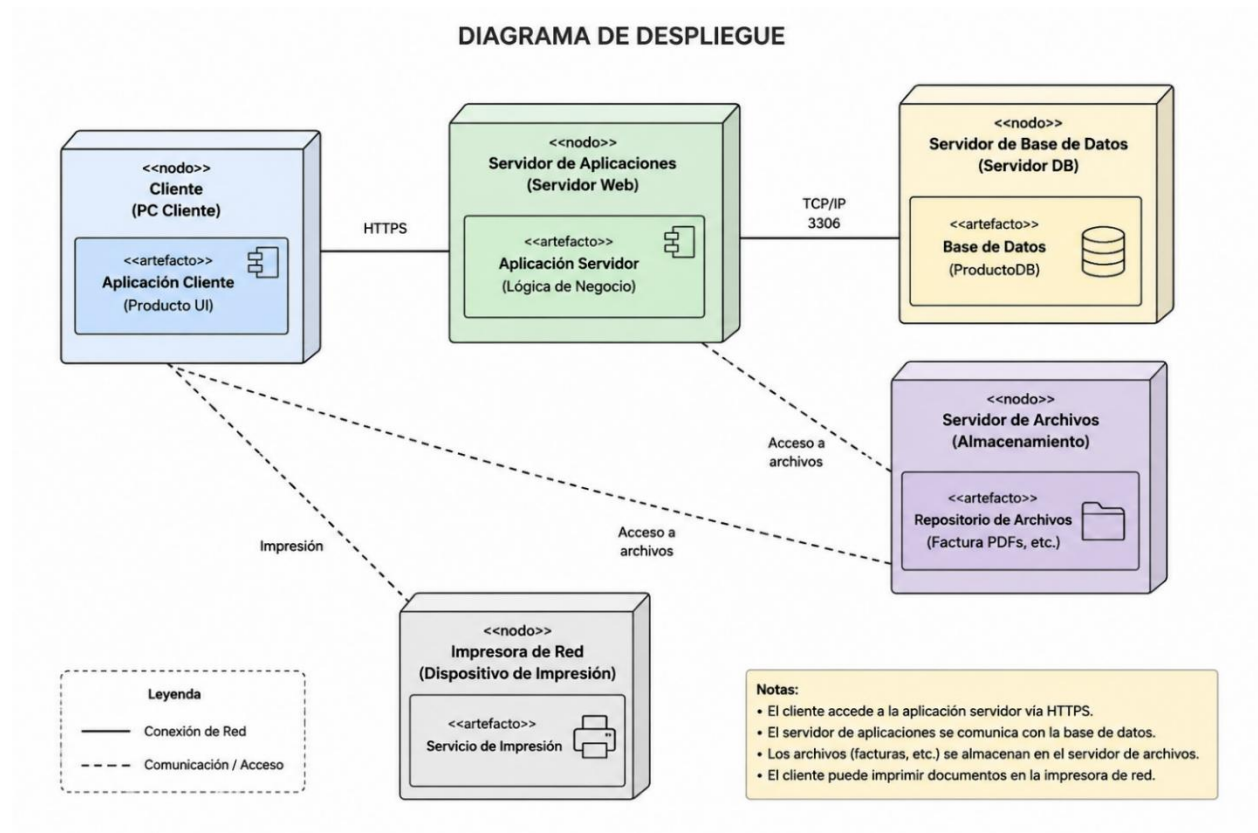
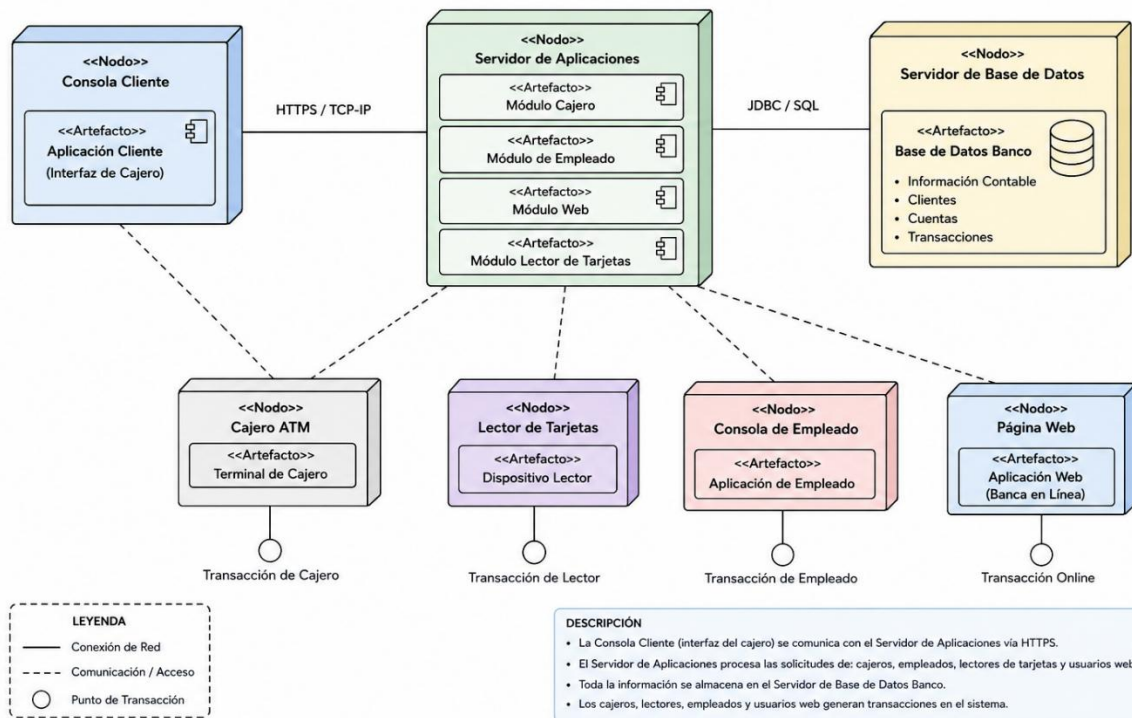


DIAGRAMA DE DESPLIEGUE



Análisis Detallado de Nodos y Conectividad

Un nodo es básicamente una unidad que puede procesar información. Puede ser un aparato físico o una creación virtual.

Tipos de Nodos

- Dispositivos físicos: servidores, computadoras de escritorio, cajas registradoras electrónicas o dispositivos integrados.
- Entornos de ejecución: sistemas operativos como Linux o Windows, servidores de aplicaciones como Tomcat o IIS, o plataformas de contenedores como Docker o Kubernetes.

Artefactos y su ubicación

Un artefacto es la forma tangible de un elemento del sistema. Por ejemplo, el elemento “Administrador de Pagos” se materializa como el archivo pagos.war dentro del nodo “Servidor de Aplicaciones”.

Conexiones entre Nodos

Es crucial definir cómo se transmiten los datos entre nodos. Al documentar, es importante detallar la seguridad (como SSL/TLS), la velocidad de transferencia y el método de comunicación (como HTTP/REST, JDBC, o AMQP para mensajería). Esto ayuda a anticipar posibles demoras y puntos débiles en la red.

3.1. Nodos y Artefactos

- Nodos: Son los recursos de ejecución, que se dividen en dispositivos (hardware físico) y entornos de ejecución (software que hospeda otros componentes).
- Artefactos: Son la manifestación física del software, como un archivo `.war`, un ejecutable `.exe` o un esquema de base de datos.

3.2. Rutas de Comunicación

El diagrama utiliza líneas de asociación para representar las conexiones físicas o lógicas entre nodos. Es fundamental especificar el protocolo de comunicación utilizado, ya que esto determina la viabilidad técnica de la arquitectura.

3.3. Profundización en Escenarios Modernos

En la era del Cloud Computing, el diagrama de despliegue ha evolucionado para representar balanceadores de carga, clusters de base de datos y redes de entrega de contenido, mejorando la escalabilidad y la disponibilidad.

4. Sinergia entre Componentes y Despliegue

La combinación de estos dos diagramas es fundamental para el diseño de ingeniería. Cada parte del diagrama de componentes debe tener un lugar en el diagrama de despliegue.

- **Asignación de Recursos:** Esto nos permite saber si un servidor es lo suficientemente poderoso para soportar los componentes que se le asignan.
- **Estrategias de Disponibilidad:** Al combinar estos diagramas, podemos crear arquitecturas que no fallen fácilmente. Por ejemplo, si tenemos una base de datos, podemos colocarla en dos nodos diferentes que se copien entre sí. De esta manera, si un nodo falla, la base de datos sigue funcionando en el otro.
- **Optimización de Latencia:** Cuando vemos ambos diagramas juntos, podemos decidir colocar el componente de caché en el mismo lugar que el servidor web. Esto reduce el tráfico de red y hace que las cosas funcionen más rápido para el usuario final.

5. Conclusión

La implementación de software de alta calidad requiere un equilibrio perfecto entre la arquitectura lógica y la infraestructura física. Los diagramas de componentes proporcionan claridad para construir sistemas modulares y cohesivos, mientras que los diagramas de despliegue garantizan que el equipo de operaciones comprenda los requisitos de hardware y conectividad para que el software sea estable en producción.

El dominio de estos modelos permite anticipar fallos, optimizar el uso de recursos y diseñar sistemas capaces de evolucionar ante las demandas cambiantes del mercado tecnológico. Sin una documentación clara de estos niveles, la implementación se convierte en un proceso errático y difícil de escalar.

Referencias:

UML Org (Oficial): <https://www.uml.org/>

Lucidchart - Guía de Diagramas de Componentes:
<https://www.lucidchart.com/pages/es/diagrama-de-componentes-uml>

Visual Paradigm - UML Deployment Diagram Tutorial: <https://www.visual-paradigm.com/guide/uml/what-is-deployment-diagram/>

IBM - Modelado de la implementación:
<https://www.ibm.com/docs/es/rsm/7.5.0?topic=topology-deployment-diagrams>